Session 14 - DOM và Event - Lý thuyết trong JavaScript

Lesson 1: Tổng quan về DOM

- Khái niệm DOM
  - DOM (Document Object Model) là một giao diện lập trình ứng dụng (API) cho các tài liệu HTML và XML.
  - Nó định nghĩa cấu trúc logic của tài liệu và cách tài liệu được truy cập và thao tác.
- DOM là cầu nối giữa HTML và JavaScript
  - JavaScript sử dụng DOM để truy cập và sửa đổi nội dung, cấu trúc và kiểu dáng của tài liệu HTML.
- Cấu trúc cây DOM
  - Tài liệu HTML được biểu diễn dưới dạng một cấu trúc cây (tree structure) các đối tượng.
  - Mỗi nút (node) trong cây đại diện cho một phần của tài liệu (ví dụ: phần tử, văn bản, thuộc tính).
- Cách JavaScript thao tác với DOM
  - JavaScript có thể:
    - Thay đổi nội dung HTML.
    - Thay đổi thuộc tính CSS.
    - Thêm hoặc xóa các phần tử HTML.
    - Phản ứng với các sự kiện của người dùng (nhấp chuột, nhập liệu).

Lesson 2: Truy xuất phần tử HTML trong DOM

- getElementById
  - Trả về phần tử đầu tiên có thuộc tính id khớp với chuỗi đã chỉ định.
  - Ví dụ: document.getElementById("myId")
- getElementsByClassName
  - Trả về một tập hợp (HTMLCollection) các phần tử có thuộc tính class khớp với chuỗi đã chỉ định.
  - Ví dụ: document.getElementsByClassName("myClass")
- getElementsByTagName
  - Trả về một tập hợp (HTMLCollection) các phần tử có tên thẻ (tag name) khớp với chuỗi đã chỉ định.
  - Ví dụ: document.getElementsByTagName("div")
- querySelector
  - Trả về phần tử đầu tiên khớp với bộ chọn CSS đã chỉ định.
  - Ví dụ: document.querySelector("#myId .myClass")
- querySelectorAll
  - Trả về một danh sách nút (NodeList) của tất cả các phần tử khớp với bộ chọn CSS đã chỉ định.
  - Ví dụ: document.querySelectorAll("p.intro")

Lesson 3: innerText, textContent và innerHTML

- innerText dùng để lấy nội dung hiển thị
  - Chỉ trả về văn bản hiển thị, bỏ qua các thẻ HTML và khoảng trắng bị ẩn bởi CSS.
  - Ví dụ: Hidden text Visible text -> innerText sẽ chỉ lấy "Visible text".
- textContent lấy toàn bộ nội dung text
  - Trả về tất cả văn bản bên trong phần tử, bao gồm cả văn bản trong các thẻ ẩn và khoảng trắng.
  - Ví dụ: Hidden text Visible text -> textContent sẽ lấy "Hidden text Visible text".
- innerHTML lấy cả HTML bên trong
  - Trả về toàn bộ nội dung HTML (bao gồm cả các thẻ HTML) bên trong phần tử.
  - Có thể dùng để lấy hoặc thiết lập nội dung HTML.
  - Ví dụ: Hello -> innerHTML sẽ lấy Hello .
- So sánh sự khác nhau
  - innerText: Chỉ văn bản hiển thị, xem xét CSS.
  - textContent: Toàn bộ văn bản, không xem xét CSS.
  - innerHTML: Toàn bộ nội dung HTML.
- Trường hợp sử dụng
  - innerText: Khi muốn hiển thị văn bản đúng như người dùng thấy.
  - textContent: Khi cần lấy tất cả văn bản thô, không quan tâm định dạng.
  - innerHTML: Khi cần lấy hoặc thay đổi cấu trúc HTML của một phần tử.

Lesson 4: setAttribute và getAttribute

- setAttribute dùng để thêm hoặc thay đổi thuộc tính
  - Cú pháp: element.setAttribute(name, value)
  - Thêm một thuộc tính mới hoặc thay đổi giá trị của một thuộc tính hiện có.
- getAttribute dùng để lấy giá trị thuộc tính
  - Cú pháp: element.getAttribute(name)
  - Trả về giá trị của thuộc tính được chỉ định.
- Ví dụ thay đổi src, href, id, class
  - Thay đổi src của ảnh: imgElement.setAttribute("src", "new_image.jpg")
  - Thay đổi href của liên kết: linkElement.setAttribute("href", "https://new_url.com")
  - Thay đổi id: divElement.setAttribute("id", "newId")
  - Thay đổi class: pElement.setAttribute("class", "newClass")

Lesson 5: Điều chỉnh style của phần tử HTML

- Thay đổi CSS bằng JavaScript
  - Sử dụng thuộc tính style của phần tử DOM để truy cập và sửa đổi các thuộc tính CSS.
- Sử dụng thuộc tính style
  - Cú pháp: element.style.propertyName = "value"
  - Tên thuộc tính CSS được viết theo kiểu camelCase (ví dụ: backgroundColor thay vì background-color).
- Ví dụ thay đổi màu sắc, kích thước, background
  - Thay đổi màu chữ: myElement.style.color = "red"
  - Thay đổi kích thước font: myElement.style.fontSize = "20px"
  - Thay đổi màu nền: myElement.style.backgroundColor = "lightblue"
  - Thay đổi chiều rộng: myElement.style.width = "100px"

Lesson 6: Sử dụng danh sách class của HTML

- classList add
  - Thêm một hoặc nhiều class vào danh sách class của phần tử.
  - Cú pháp: element.classList.add("class1", "class2")
- classList remove
  - Xóa một hoặc nhiều class khỏi danh sách class của phần tử.
  - Cú pháp: element.classList.remove("classToRemove")
- classList toggle
  - Nếu class tồn tại, nó sẽ bị xóa; nếu không tồn tại, nó sẽ được thêm vào.
  - Cú pháp: element.classList.toggle("toggleClass")
- classList contains
  - Kiểm tra xem phần tử có chứa một class cụ thể hay không.
  - Trả về true hoặc false.
  - Cú pháp: element.classList.contains("className")

Lesson 7: DOM traversal

- parentElement
  - Trả về phần tử cha trực tiếp của phần tử hiện tại.
- children
  - Trả về một HTMLCollection của tất cả các phần tử con của phần tử hiện tại.
- firstElementChild
  - Trả về phần tử con đầu tiên của phần tử hiện tại.
- lastElementChild
  - Trả về phần tử con cuối cùng của phần tử hiện tại.
- nextElementSibling
  - Trả về phần tử anh chị em tiếp theo (cùng cấp) của phần tử hiện tại.
- previousElementSibling
  - Trả về phần tử anh chị em trước đó (cùng cấp) của phần tử hiện tại.

Lesson 8: Append child và remove

- createElement
  - Tạo một phần tử HTML mới với tên thẻ được chỉ định.
  - Cú pháp: document.createElement("tagName")
- appendChild
  - Thêm một nút làm con cuối cùng của nút cha được chỉ định.
  - Cú pháp: parentNode.appendChild(childNode)
- append
  - Thêm một nút hoặc chuỗi văn bản vào cuối danh sách con của một phần tử.
  - Hỗ trợ thêm nhiều đối số (nút hoặc chuỗi).
  - Cú pháp: parentNode.append(child1, "text", child2)
- removeChild
  - Xóa một nút con được chỉ định khỏi danh sách con của nút cha.
  - Cú pháp: parentNode.removeChild(childNode)
- remove
  - Xóa chính phần tử đó khỏi DOM.
  - Cú pháp: element.remove()

Lesson 9: Tổng quan về event

- Khái niệm event
  - Event (sự kiện) là một hành động hoặc một sự kiện xảy ra trong hệ thống mà trình duyệt hoặc JavaScript có thể phản ứng lại.
  - Ví dụ: nhấp chuột, nhấn phím, tải trang, gửi form.
- Event object
  - Khi một sự kiện xảy ra, một đối tượng Event được tạo ra.
  - Đối tượng này chứa thông tin chi tiết về sự kiện (ví dụ: vị trí con trỏ chuột, phím được nhấn).
- Các loại sự kiện phổ biến
  - **Mouse events**: click, mouseover, mouseout, mousedown, mouseup.
  - **Keyboard events**: keydown, keyup, keypress.
  - **Form events**: submit, change, focus, blur.
  - **Document/Window events**: load, unload, resize, scroll.

Lesson 10: Gắn sự kiện cho phần tử HTML

- Inline event trong HTML
  - Gắn trình xử lý sự kiện trực tiếp vào thuộc tính HTML.
  - Ví dụ: <button onclick="myFunction()">Click me</button>
  - **Nhược điểm**: Trộn lẫn HTML và JavaScript, khó quản lý.
- Gán sự kiện bằng thuộc tính onevent
  - Gán một hàm xử lý sự kiện cho thuộc tính onEvent của phần tử DOM.
  - Ví dụ: myButton.onclick = myFunction;
  - **Nhược điểm**: Chỉ có thể gán một trình xử lý cho mỗi sự kiện.
- addEventListener
  - Phương pháp khuyến nghị để gán trình xử lý sự kiện.
  - Cho phép gắn nhiều trình xử lý sự kiện cho cùng một sự kiện trên cùng một phần tử.
  - Cú pháp: element.addEventListener("eventType", handlerFunction, useCapture)
- So sánh các cách gắn sự kiện
  - **Inline**: Đơn giản nhưng không nên dùng cho ứng dụng lớn.
  - **onevent property**: Chỉ một trình xử lý, dễ ghi đè.
  - **addEventListener**: Linh hoạt nhất, cho phép nhiều trình xử lý, hỗ trợ capturing/bubbling.

Lesson 11: Mouse và Keyboard event

- click
  - Xảy ra khi người dùng nhấp chuột vào một phần tử.
  - Ví dụ: button.addEventListener("click", () => console.log("Button clicked!"))
- mouseover và mouseout
  - mouseover: Xảy ra khi con trỏ chuột di chuyển vào một phần tử.
  - mouseout: Xảy ra khi con trỏ chuột di chuyển ra khỏi một phần tử.
  - Ví dụ: div.addEventListener("mouseover", () => div.style.backgroundColor = "yellow")
- keydown
  - Xảy ra khi người dùng nhấn một phím.
  - Kích hoạt lặp lại nếu giữ phím.
  - Ví dụ: document.addEventListener("keydown", (event) => console.log("Key pressed:", event.key))
- keyup
  - Xảy ra khi người dùng nhả một phím.
  - Ví dụ: document.addEventListener("keyup", (event) => console.log("Key released:", event.key))

Lesson 12: Form submit event

- submit event
  - Xảy ra khi một form được gửi.
  - Được kích hoạt trên phần tử <form>.
  - Ví dụ: myForm.addEventListener("submit", (event) => { /* handle submit */ })
- preventDefault
  - Phương thức của đối tượng Event dùng để ngăn chặn hành vi mặc định của trình duyệt.
  - Thường được sử dụng trong sự kiện submit để ngăn form gửi dữ liệu và tải lại trang.
  - Ví dụ: event.preventDefault()
- Kiểm tra dữ liệu form bằng JavaScript
  - Trong trình xử lý sự kiện submit, có thể truy cập các trường form và thực hiện kiểm tra dữ liệu (validation).
  - Nếu dữ liệu không hợp lệ, gọi event.preventDefault() và hiển thị thông báo lỗi.
  - Ví dụ: javascript myForm.addEventListener("submit", (event) => { const username = document.getElementById("username").value; if (username.length < 5) { alert("Username must be at least 5 characters long."); event.preventDefault(); // Ngăn form gửi } });

Presented with xmind